

ГУАП

КАФЕДРА № 44

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

канд. техн. наук,
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

Генерация кода по диаграмме классов

по курсу: Технология программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4341В

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2025

1 Исходная диаграмма классов из лабораторной работы № 3



Рисунок 1 - Исходная диаграмма классов

2 Диаграмма классов в пакете StarUML

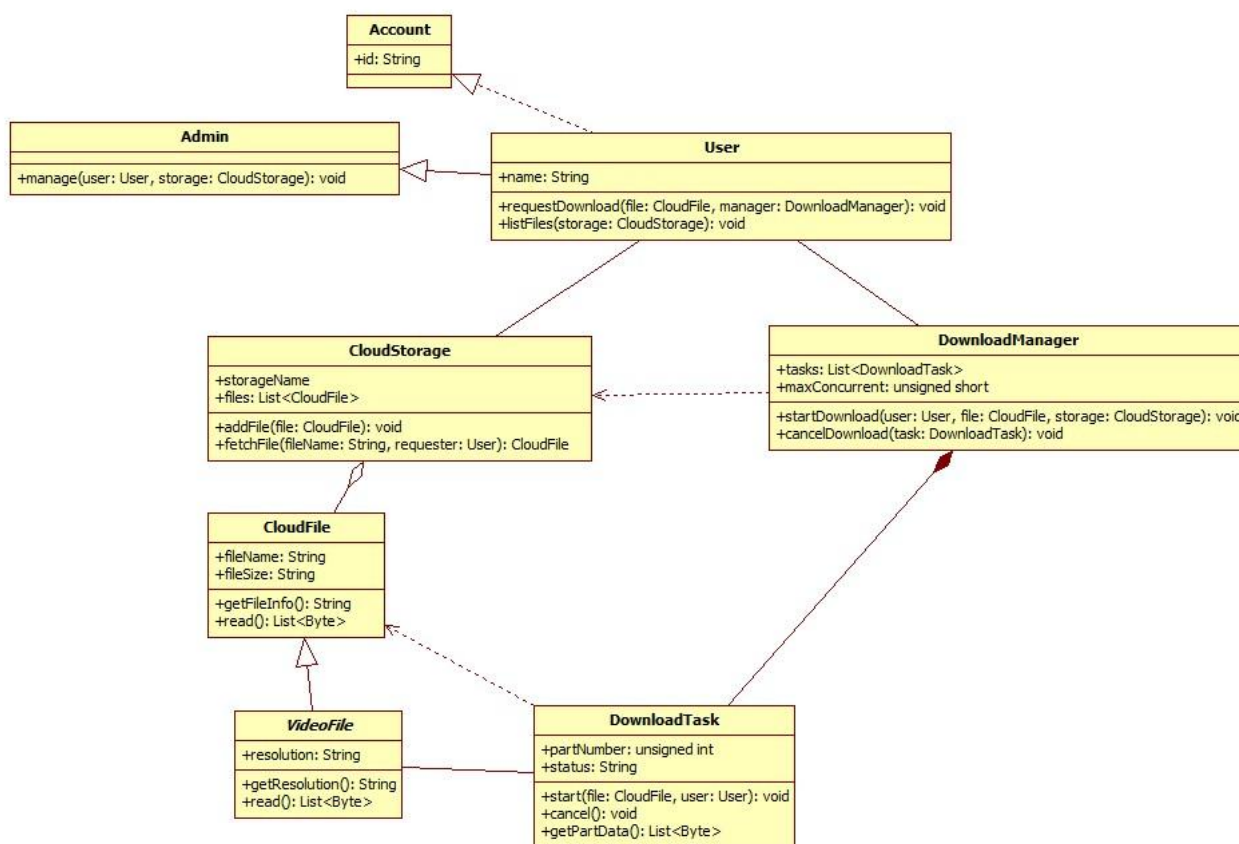


Рисунок 2 - Диаграмма классов в пакете StarUML

3 Описание технологии формирования кода со скриншотами

Для выполнения автоматической генерации программного кода на языке C++ в пакете StarUML необходимо предварительно активировать соответствующий профиль языка. Для этого в главном меню программы в верхней части экрана выбирается пункт Model, после чего открывается раздел Profiles (Рис 3). В появившемся окне из списка доступных профилей выбирается профиль C++ Profile, который затем подключается к текущей модели с помощью команды Include. После подключения профиля окно профилей закрывается.

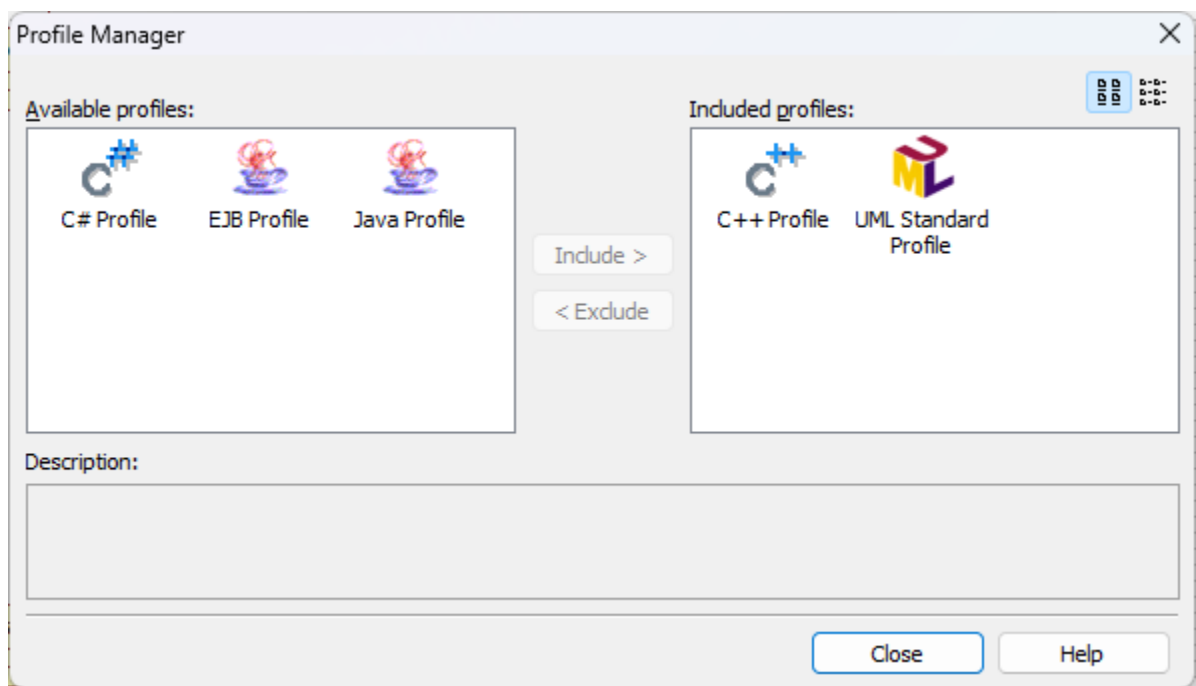


Рисунок 3 – Добавление профиля C++

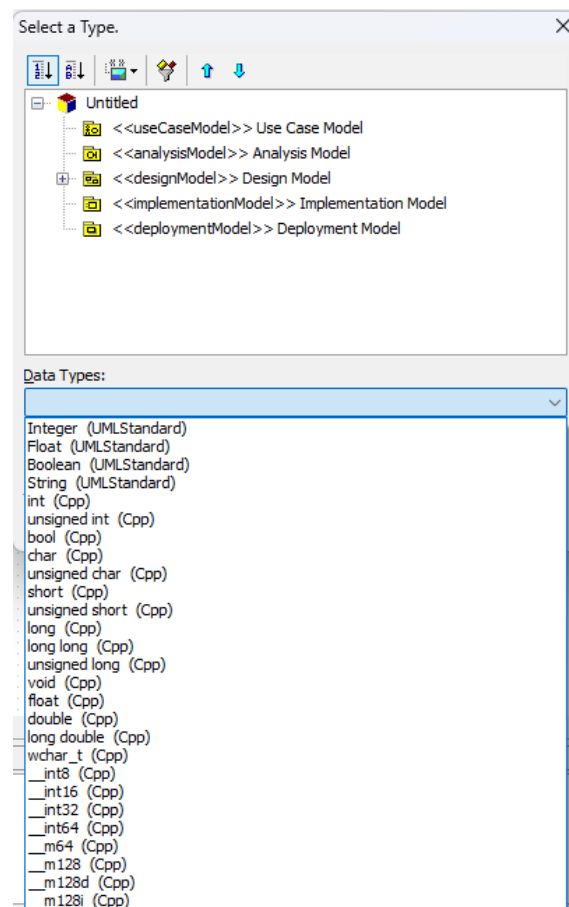


Рисунок 4 – Список типов данных

Далее осуществляется переход к панели инструментов StarUML, где становится доступен набор средств, относящихся к языку C++. В данном разделе выбирается пункт C++, после чего используется команда Generate Code (Рис 5), предназначенная для автоматического формирования программного кода на основе диаграммы классов.

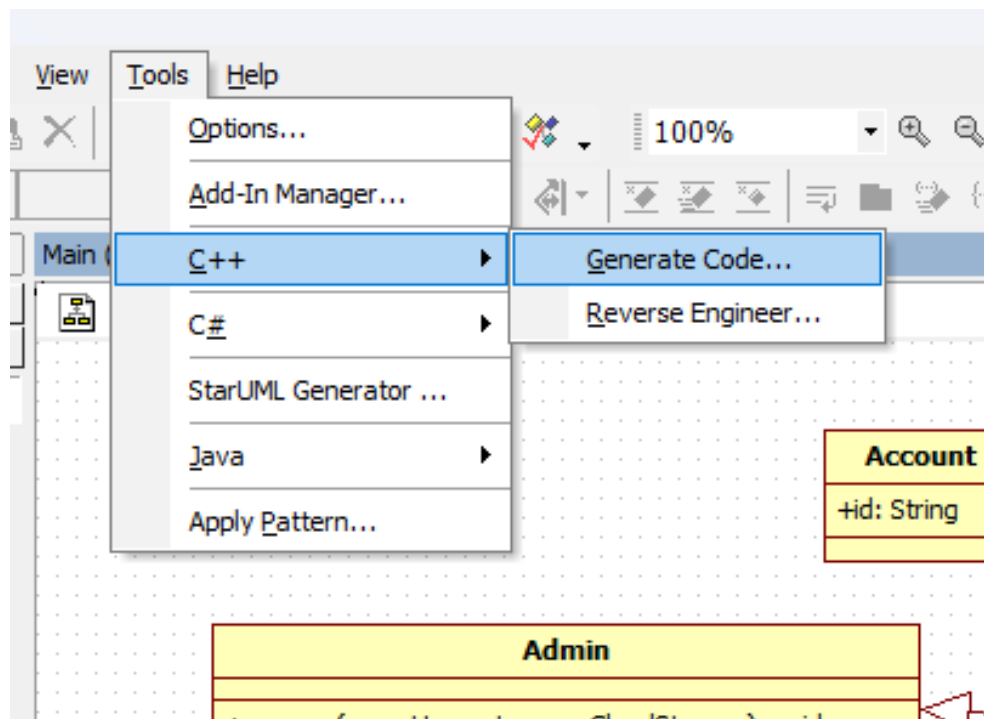


Рисунок 5 - Команда Generate Code

На следующем этапе предлагается выбрать модель, по которой будет выполняться генерация. В качестве исходной модели указывается диаграмма классов, разработанная ранее. После этого в диалоговом окне генерации необходимо отметить все элементы модели, для которых требуется создать программный код, включая классы, их атрибуты и методы (Рис 6).

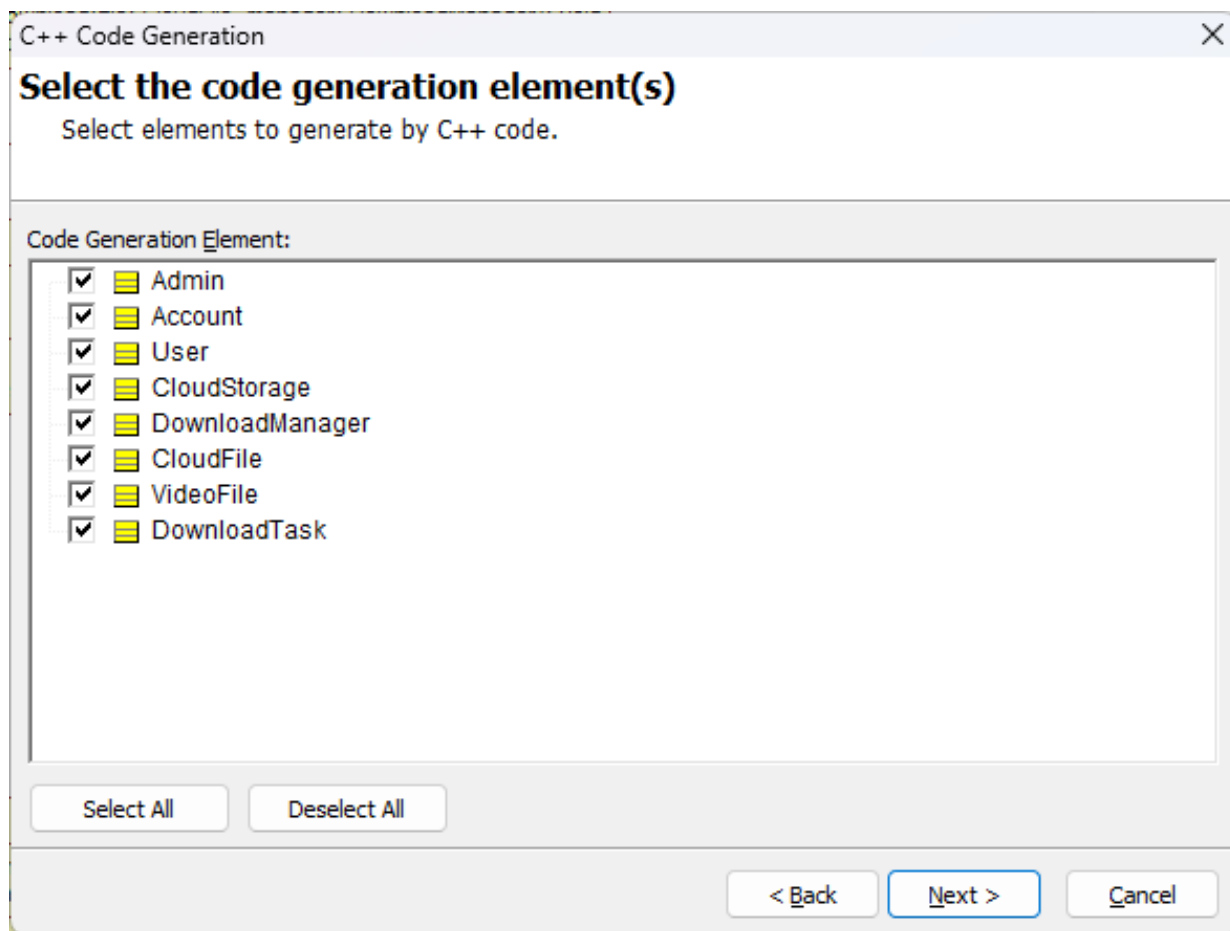


Рисунок 6 – Выбор элементов для генерации

После подтверждения всех настроек процесс автоматической генерации запускается, и StarUML формирует программный код на языке C++ в соответствии со структурой диаграммы классов. В результате пользователь получает набор файлов, отражающих структуру проектируемой системы и готовых для дальнейшей доработки и реализации логики программы (Рис 7-8).

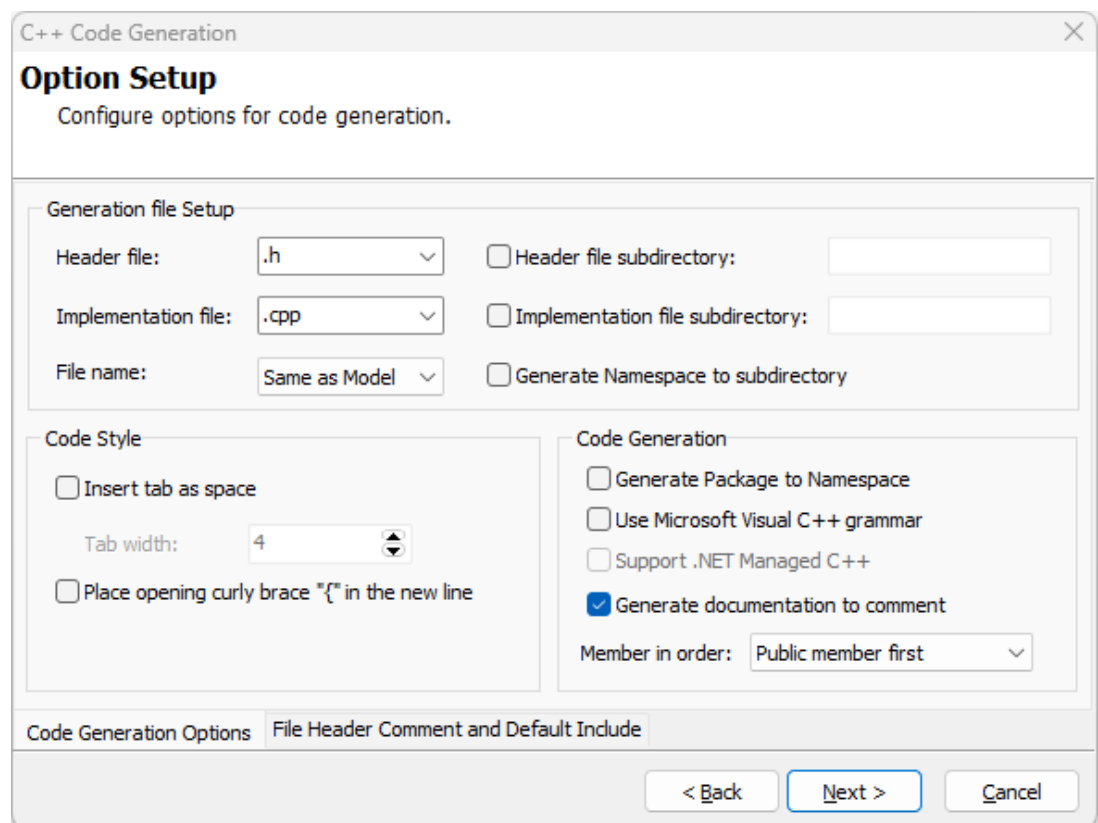


Рисунок 7 – Конфигурация генерации

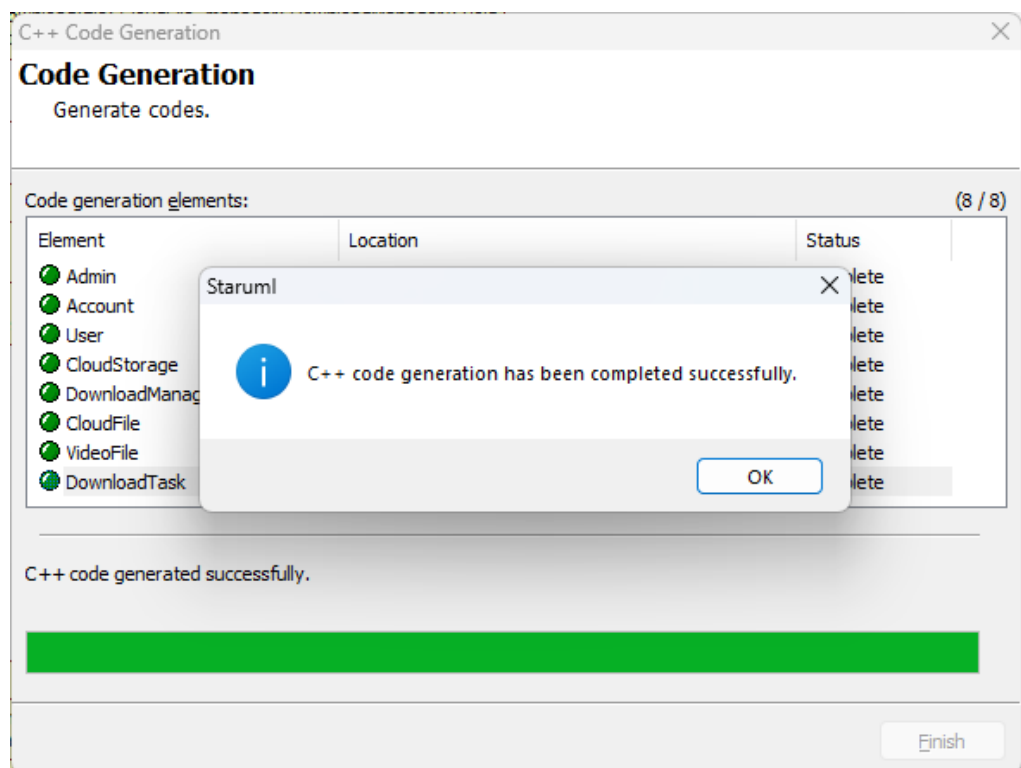


Рисунок 8 – Успешная генерация

4 Сгенерированный код

```
//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : Account.h
//  @ Date : 19.12.2025
//  @ Author :
//
//

#ifdef _ACCOUNT_H
#define _ACCOUNT_H

class Account {
public:
    String id;
};

#endif // _ACCOUNT_H


//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : Admin.cpp
//  @ Date : 19.12.2025
//  @ Author :
//
//

#include "Admin.h"
#include "User.h"
#include "CloudStorage.h"

void Admin::manage(User user, CloudStorage storage) {

}


//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : Admin.h
//  @ Date : 19.12.2025
//  @ Author :
//
//
```



```

//

#if !defined(_ADMIN_H)
#define _ADMIN_H

#include "User.h"
#include "CloudStorage.h"

class Admin {
public:
    void manage(User user, CloudStorage storage);
};

#endif // _ADMIN_H


//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : CloudFile.cpp
//  @ Date : 19.12.2025
//  @ Author :
//
//

#include "CloudFile.h"

String CloudFile::getFileInfo() {

}

List<Byte> CloudFile::read() {

}


//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : CloudFile.h
//  @ Date : 19.12.2025
//  @ Author :
//
//

#if !defined(_CLOUDFILE_H)
#define _CLOUDFILE_H

class CloudFile {
public:

```

```

        String fileName;
        String fileSize;
        String getFileInfo();
        List<Byte> read();
};

#endif // _CLOUDFILE_H

//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : CloudStorage.cpp
//  @ Date : 19.12.2025
//  @ Author :
//
//

#include "CloudStorage.h"
#include "CloudFile.h"
#include "User.h"

void CloudStorage::addFile(CloudFile file) {

}

CloudFile CloudStorage::fetchFile(String fileName, User requester) {

}

//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : CloudStorage.h
//  @ Date : 19.12.2025
//  @ Author :
//
//

#if !defined(_CLOUDSTORAGE_H)
#define _CLOUDSTORAGE_H

#include "CloudFile.h"
#include "User.h"

class CloudStorage {
public:
    int storageName;
    List<CloudFile> files;
    void addFile(CloudFile file);
    CloudFile fetchFile(String fileName, User requester);
};

```

```

};

#endif // _CLOUDSTORAGE_H

//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : DownloadManager.cpp
//  @ Date : 19.12.2025
//  @ Author :
//
//

#include "DownloadManager.h"
#include "User.h"
#include "CloudFile.h"
#include "CloudStorage.h"
#include "DownloadTask.h"

void DownloadManager::startDownload(User user, CloudFile file, CloudStorage
storage) {

}

void DownloadManager::cancelDownload(DownloadTask task) {

}

//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : DownloadManager.h
//  @ Date : 19.12.2025
//  @ Author :
//
//

#if !defined(_DOWNLOADMANAGER_H)
#define _DOWNLOADMANAGER_H

#include "User.h"
#include "CloudFile.h"
#include "CloudStorage.h"
#include "DownloadTask.h"

class DownloadManager {
public:
    List<DownloadTask> tasks;
    unsigned short maxConcurrent;
    void startDownload(User user, CloudFile file, CloudStorage storage);

```

```

        void cancelDownload(DownloadTask task);
};

#endif // _DOWNLOADMANAGER_H

//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : DownloadTask.cpp
//  @ Date : 19.12.2025
//  @ Author :
//
//

#include "DownloadTask.h"
#include "CloudFile.h"
#include "User.h"

void DownloadTask::start(CloudFile file, User user) {

}

void DownloadTask::cancel() {

}

List<Byte> DownloadTask::getPartData() {

}

//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : DownloadTask.h
//  @ Date : 19.12.2025
//  @ Author :
//
//

#if !defined(_DOWNLOADTASK_H)
#define _DOWNLOADTASK_H

#include "CloudFile.h"
#include "User.h"

class DownloadTask {
public:
    unsigned int partNumber;
    String status;
    void start(CloudFile file, User user);
};

```

```

        void cancel();
        List<Byte> getPartData();
};

#endif // _DOWNLOADTASK_H

//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : User.cpp
//  @ Date : 19.12.2025
//  @ Author :
//
//

#include "User.h"
#include "CloudFile.h"
#include "DownloadManager.h"
#include "CloudStorage.h"

void User::requestDownload(CloudFile file, DownloadManager manager) {

}

void User::listFiles(CloudStorage storage) {

}

//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : User.h
//  @ Date : 19.12.2025
//  @ Author :
//
//

#if !defined(_USER_H)
#define _USER_H

#include "Admin.h"
#include "Account.h"
#include "CloudFile.h"
#include "DownloadManager.h"
#include "CloudStorage.h"

class User : public Admin, public Account {
public:
    String name;
    void requestDownload(CloudFile file, DownloadManager manager);

```

```

        void listFiles(CloudStorage storage);
};

#endif // _USER_H

//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : VideoFile.cpp
//  @ Date : 19.12.2025
//  @ Author :
//
//

#include "VideoFile.h"

String VideoFile::getResolution() {

}

List<Byte> VideoFile::read() {

}

//
//
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Untitled
//  @ File Name : VideoFile.h
//  @ Date : 19.12.2025
//  @ Author :
//
//

#if !defined(_VIDEOFILE_H)
#define _VIDEOFILE_H

#include "CloudFile.h"

class VideoFile : public CloudFile {
public:
    String resolution;
    String getResolution();
    List<Byte> read();
};

#endif // _VIDEOFILE_H

```

5 Описание технологии генерации диаграммы классов на основе кода

Для выполнения обратного проектирования и автоматической генерации диаграммы классов по уже существующему программному коду в пакете StarUML создаётся новая модель. После этого осуществляется переход во вкладку Tools, где выбирается раздел C++, а затем команда Reverse Engineer (Рис 9), предназначенная для построения UML-диаграммы на основе исходных файлов программы.

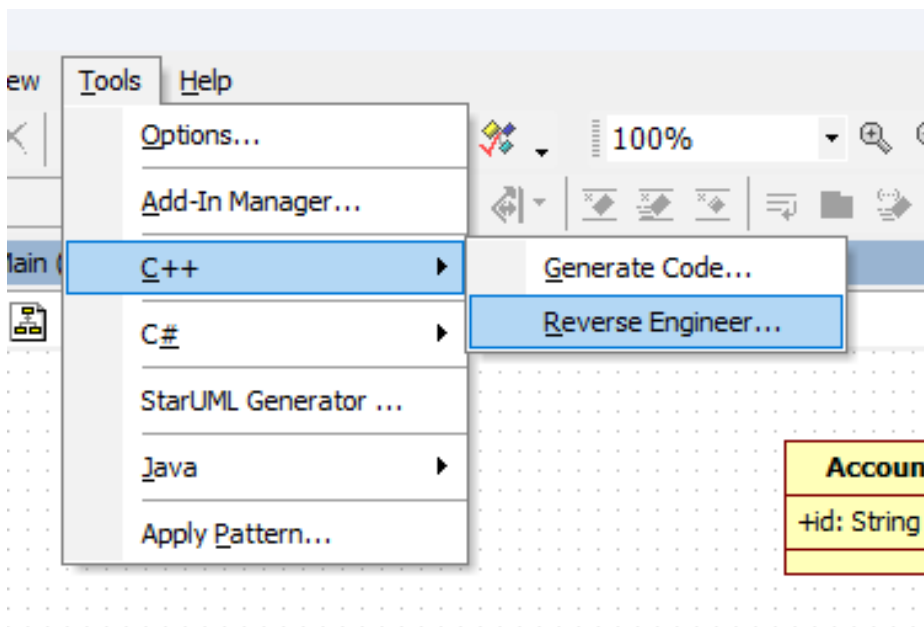


Рисунок 9 – Команда Reverse Engineer

В открывшемся окне пользователю необходимо указать каталог, содержащий файлы исходного кода на языке C++ (Рис 10). После выбора папки из списка файлов выделяются те исходные и заголовочные файлы, которые должны участвовать в обратном проектировании. Выбранные файлы добавляются в список обрабатываемых элементов с помощью соответствующей команды, после чего осуществляется переход к следующему этапу настройки.

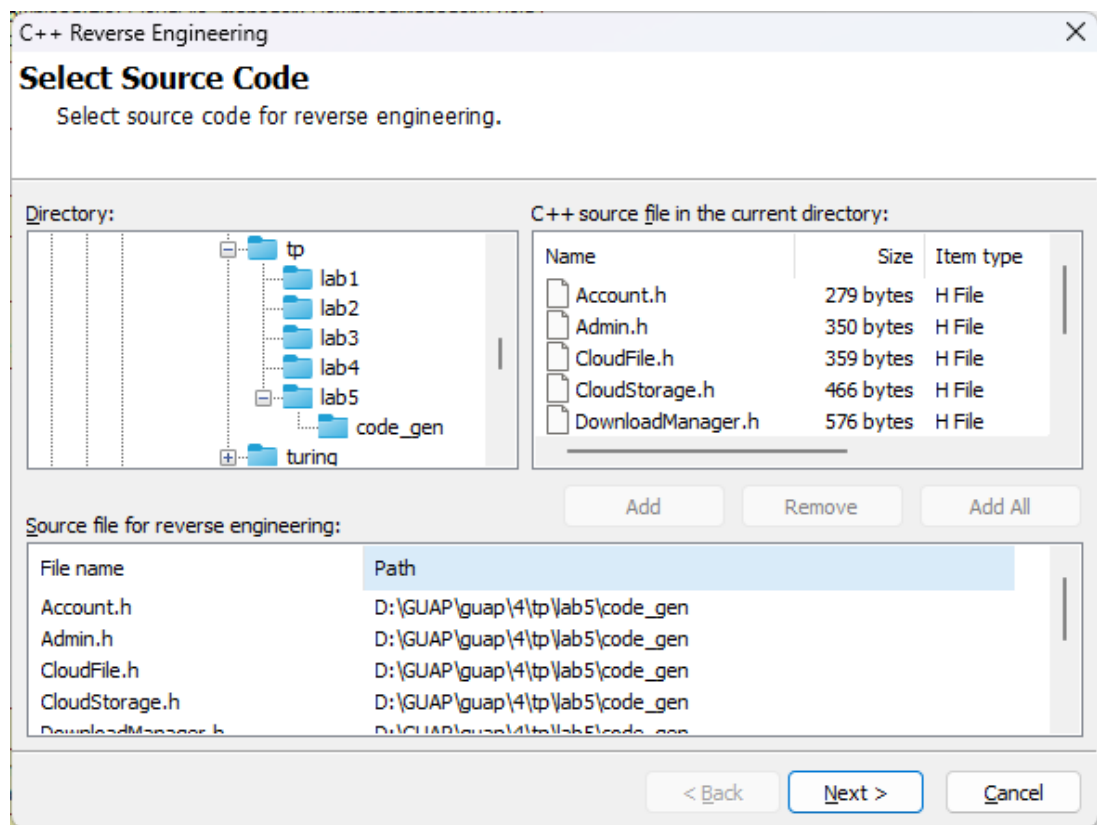
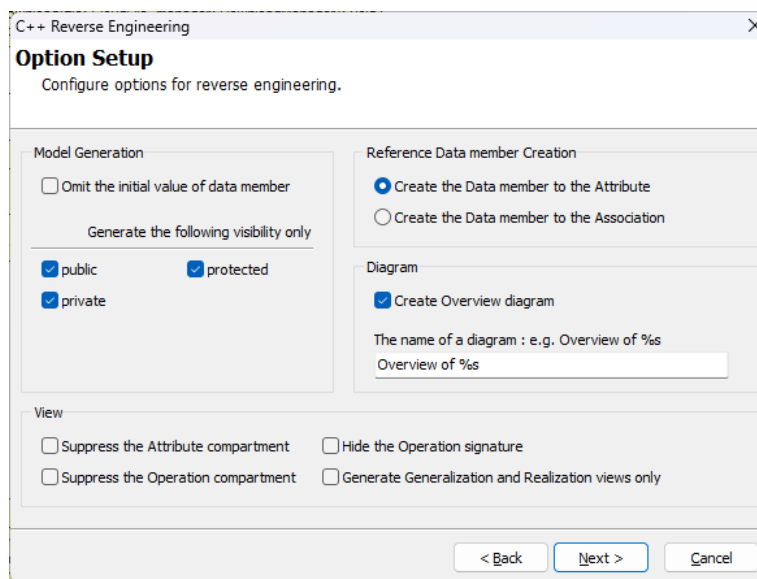


Рисунок 10 – Выбор файлов с исходным кодом

После выбора модели открывается окно дополнительных параметров обратного проектирования. Особое значение имеет настройка параметра, определяющего способ формирования связей между классами. В зависимости от выбранного варианта итоговый вид диаграммы может отличаться. При выборе опции, предусматривающей создание атрибутов на основе ссылок на другие классы, элементы взаимодействия между классами отображаются в виде атрибутов соответствующих типов. В этом случае после завершения генерации все классы и их элементы автоматически добавляются в модель и отображаются в области проекта.



Если же в параметрах обратного проектирования выбрать вариант формирования связей в виде ассоциаций между классами, то связи между объектами будут представлены непосредственно в виде UML-ассоциаций, а не атрибутов.

В моем случае оба варианта дают один и тот же эффект.

6 Диаграмма классов, полученная в результате обратного проектирования

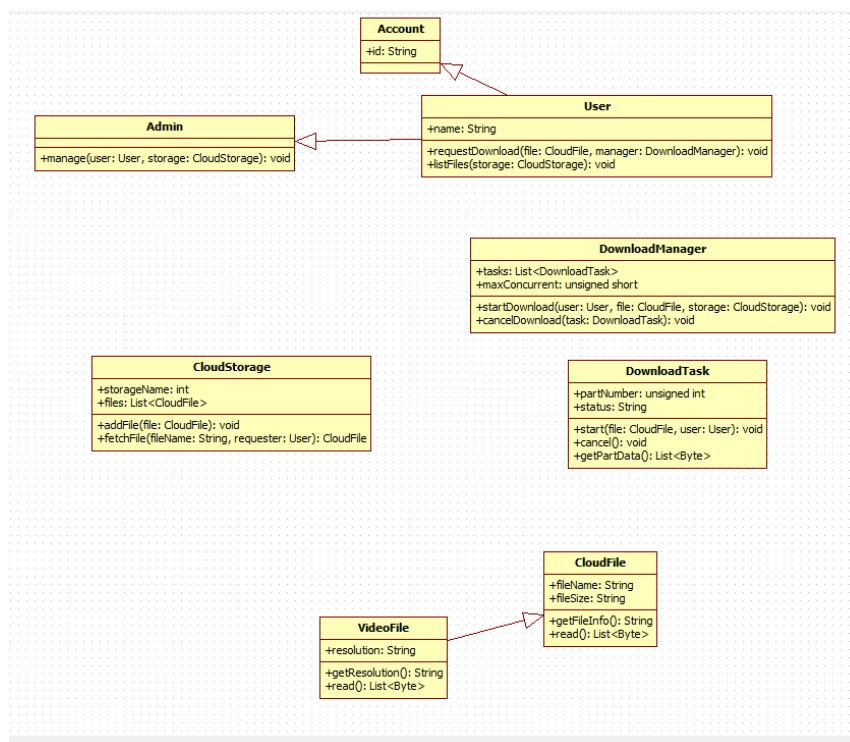


Рисунок 11 – Диаграмма классов при обратном проектировании

7 Анализ полученных результатов

В ходе выполнения лабораторной работы были изучены возможности прямого и обратного проектирования с использованием UML-диаграмм классов и языка программирования C++. При генерации кода на основе диаграммы классов было установлено, что полученный программный код в целом формируется корректно: сохраняются модификаторы доступа, структура классов, атрибуты и методы, а также создаются заголовочные файлы и файлы реализации для каждого класса. Методы представлены в виде корректных заглушек, пригодных для последующей ручной доработки.

Вместе с тем выявлены ограничения автоматической генерации. В частности, не во всех случаях в заголовочные файлы добавляются директивы `include` для классов, используемых в качестве параметров методов или атрибутов, если они не заданы по ссылке или указателю. Это может приводить к потере информации о связях между классами и требует ручного исправления сгенерированного кода. При этом корректно заданные типы данных переносятся без изменений.

При выполнении обратного проектирования было установлено, что восстановление диаграммы классов из кода осуществляется менее точно. В зависимости от выбранных настроек генерации часть связей может быть представлена в виде атрибутов, либо не восстановлена вовсе.

Таким образом, сделан вывод, что прямое проектирование из UML в код работает удовлетворительно, тогда как обратное проектирование из кода в диаграмму требует внимательного анализа и ручной корректировки, поскольку автоматические средства не всегда полностью и однозначно восстанавливают исходную структуру модели.